16-720 Computer Vision

Assignment 3: Geometry in Motion

Fall 2026 Released October 30, 2026

This handout has every question, requirement, and grading rule. The README explains setup, and the starter notebook holds the code you run for each part. Read the full handout before starting.

Item	Requirement
Release	October 30, 2026
Due	November 24, 2026, 11:59 pm ET (Gradescope)
Base score	100 points
Optional credit	Up to 10 points total, including the Slack showcase
Expected time	10+ hours
Runtime	Google Colab. Required work runs on CPU.
Submit	Python code, executed notebook, written PDF, Hero Result, and still image (Section 12)

2. Overview

In this assignment you estimate how a flat surface moves between images and learn when that estimate can be trusted. Part A derives the equations behind homography estimation and local image alignment. Part B implements them. Part C uses them to track a planar target in video you record yourself, and Part D tests where the method breaks and why.

The course supplies feature detection, descriptor matching, image pyramids, interpolation, warping, video input and output, and plotting. You write the estimation and alignment code that the rest of the pipeline relies on.

Each part heading gives a time estimate for the work in that part. Plan extra time for setup and debugging.

3. Learning objectives

After completing this assignment, you should be able to:

- derive the linear constraints used to estimate a planar projective transformation
- normalize image coordinates and solve homogeneous least-squares problems with SVD
- implement and diagnose RANSAC for robust homography estimation
- derive and solve the Lucas-Kanade brightness-constancy system
- tell apart geometric error, photometric error, and how good a result looks
- explain the assumptions behind homographies and local image alignment
- connect projective geometry to camera pose, epipolar geometry, stereo, and reconstruction
- show where a method works and where it fails in a clear visual result

4. Academic integrity and generative AI

Work	Policy
Part A	Generative AI may not generate, explain, solve, check, or rewrite your answers.
Part B required functions	Generative AI may not generate, complete, translate, explain, debug, or rewrite the five required functions in `student/pa3.py`.
Parts C, D, and E	Generative AI is allowed and encouraged. Disclose how you used it and make sure you understand the result.

Working with others. Talk to your classmates. Discussing derivations, debugging strategies, and experiment design with other students is one of the best ways to learn this material, and so is coming to office hours early. Share ideas, not work. Everything you submit, including derivations, code, figures, and text, must be your own. Do not read or copy another student's solutions, and do not let anyone copy yours. List everyone you worked with in your acknowledgments. Collaboration is never penalized, but unacknowledged copying is an integrity violation.

You may always use course notes, textbooks, papers, ordinary web resources, library documentation, and course staff. Do not call an OpenCV homography estimator inside a required Part B function. Acknowledge every collaborator, outside resource, dataset, pretrained model, and use of AI.

5. Shared conventions and allowed tools

- Point arrays have shape $(N, 2)$, order (x, y), and dtype `float64`.
- Images are (H, W) or (H, W, C) and are indexed `image[y, x]`.
- Pixel centers are at integer coordinates.
- A homography maps source points to destination points: $\mathbf{x}_{\text{dst}} \sim H\mathbf{x}_{\text{src}}$.
- Warping samples the source image at inverse-mapped coordinates. Interpolation and validity checks are separate steps.
- A point with an invalid homogeneous scale is marked invalid, not clipped.
- No required function changes its inputs or computes an explicit matrix inverse where a linear solve works.
- The random seed is 16720. All required code runs on a free Colab CPU runtime.

All required functions are in `student/pa3.py`. Do not change their names, arguments, or return values. The constants `RANSAC_MAX_TRIALS` $= 5000$, `LK_MAX_ITERS` $= 50$, `LK_TOL` $= 10^{-3}$, `LK_MIN_VALID` $= 25$, and `LK_MAX_CONDITION` $= 10^6$ are part of this interface. Import them instead of retyping the values.

6. Part A (Think): Constraints, Estimation, and Motion (25 points, 1.5 to 2 hours)

Complete Part A without generative AI. Typeset all answers in LaTeX and show your work.

6.1 A1: Derive the DLT constraints (9 points)

Let corresponding image points be $\mathbf{x} = [x, y, 1]^T$ and $\mathbf{x}' = [x', y', 1]^T$, related by a homography $\mathbf{x}' \sim H\mathbf{x}$, where

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}.$$

Unlike the row-major ordering used in lecture, this problem uses the **column-major** vectorization

$$\mathbf{h} = \text{vec}_c(H) = [h_{11}, h_{21}, h_{31}, h_{12}, h_{22}, h_{32}, h_{13}, h_{23}, h_{33}]^T.$$

Use this ordering throughout A1 and in your code. Do not transpose H. Only the vectorization order changes.

1. Starting from homogeneous equality up to scale, eliminate the scale and derive the two independent scalar equations that one correspondence contributes.
2. Collect coefficients according to $\text{vec}_c(H)$ and write the resulting 2×9 matrix A_i satisfying $A_i\mathbf{h} = \mathbf{0}$.
3. Let $\mathbf{h}_r = \text{vec}_r(H)$ be the row-major vector used in lecture, and define a permutation matrix P by $\mathbf{h}_r = P\mathbf{h}$. Without listing all 81 entries of P, state how its nonzero positions are determined, and derive the relationship between the column-major design matrix A and the lecture's row-major design matrix A_r.
4. State the minimum number of point correspondences needed in the nondegenerate case, explain why a homography has eight degrees of freedom rather than nine, and explain why four collinear correspondences are degenerate.
5. For noisy data, write the total least-squares problem and explain why the right singular vector of A with the smallest singular value is the DLT solution. Explain how to reshape this vector into H without accidentally transposing it.

6.2 A2: Normalize a small correspondence set (5 points)

Consider image points $(0, 0)$, $(2, 0)$, $(2, 2)$, and $(0, 2)$.

1. Compute their centroid and their mean Euclidean distance from the centroid.
2. Construct a similarity transform T that moves the centroid to the origin and makes the mean distance $\sqrt{2}$.
3. Transform all four points explicitly.
4. If normalized DLT estimates $\tilde{H}$ from $\tilde{\mathbf{x}} = T\mathbf{x}$ and $\tilde{\mathbf{x}}' = T'\mathbf{x}'$, derive the expression that converts $\tilde{H}$ back to image coordinates.
5. Explain why coordinate normalization improves the numerical conditioning of the DLT system. Does it make least-squares DLT robust to mismatched correspondences? Justify your answer.

6.3 A3: RANSAC sampling without replacement (5 points)

The lecture analysis treats the points in a minimal sample as independent draws with a fixed inlier probability. For this problem, assume instead that each RANSAC trial picks s correspondences uniformly without replacement. The data contain exactly M correspondences, and exactly I of them are inliers.

1. Write the exact probability q that one sample of size s contains only inliers. Use binomial coefficients or a product that makes sampling without replacement explicit.
2. Use the following stopping-budget formula (you do not need to derive it):

$$N = \left\lceil \frac{\log(1-p)}{\log(1-q)} \right\rceil .$$

   For homography estimation, compute N for $M = 40$, $I = 20$, $s = 4$, and $p = 0.99$. Compare it with the budget from the lecture approximation $q_{\text{iid}} = (I/M)^s$, and briefly explain why the difference goes in that direction.
3. After several trials, the best model has 26 inliers among the same 40 correspondences. Should an adaptive RANSAC implementation increase or decrease its stopping budget? Explain. Why could an overly optimistic consensus count make it stop too early?
4. Give one example where RANSAC returns a homography with high consensus that is still the wrong model for the intended scene.

6.4 A4: Local motion with additive brightness change (6 points)

Suppose a small patch moves by (u, v) between two frames and its brightness changes by an unknown constant β:

$$I(x + u, y + v, t + 1) = I(x, y, t) + \beta.$$

1. Use a first-order Taylor approximation to derive $I_x u + I_y v - \beta = -I_t$. Explain how ordinary brightness constancy follows as a special case.
2. Explain why one pixel cannot determine all three unknowns. State what spatial variation a patch needs for the joint system to be well constrained.
3. For a patch with gradient rows $(I_x, I_y) = (1, 0), (0, 1), (1, 1)$ and temporal derivatives $I_t = -1, -2, -4$, construct $A\boldsymbol{\theta} = \mathbf{b}$ for $\boldsymbol{\theta} = [u, v, \beta]^T$ and solve it exactly.
4. Describe what the eigenvalues and conditioning of $A^T A$ tell you about ambiguity between horizontal motion, vertical motion, and a brightness change.
5. Predict which method should fail first as the motion between frames grows: single-scale Lucas-Kanade refinement or descriptor matching with RANSAC. You will revisit this prediction in D1.

7. Part B (Build): Robust Planar Motion (35 points, 2.5 to 3 hours)

You may use NumPy, the supplied helpers, OpenCV and SciPy documentation, and course materials. Generative AI may not write the required implementations. The starter code provides image input and output, feature detection and description, candidate matching, plotting, test data, video decoding, and Colab setup.

7.1 B1: Read the geometry pipeline (5 points)

Before implementing anything, read the numbered cells of the starter notebook and answer B1.1–B1.3 and B1.5. Answer B1.4 after implementing B2. Each answer must name a specific array shape, coordinate convention, or step in the code, not only a general definition.

1. **B1.1** In the coordinate-convention cell, what shapes do `xy` and `to_homogeneous(xy)` have, which columns hold x and y, and why does the projection transpose the row array around the matrix multiplication?
2. **B1.2** The supplied warp takes `H_src_to_dst`. Which direction does that matrix map, which direction is solved for each destination pixel, and why is that not a contradiction?
3. **B1.3** What are the two outputs of `bilinear_sample`, and why must Lucas-Kanade combine validity masks instead of treating interpolation as a validity test?
4. **B1.4** After implementing B2, identify the two design-matrix rows in your DLT implementation that match your A1 equations. Why must the recovered vector be reshaped with `order="F"`?
5. **B1.5** The pyramid list stores full resolution at index zero. Trace a full-resolution displacement of eight pixels down to level two and back.

7.2 B2: Normalized DLT (10 points)

Implement the three estimation functions:

- `normalize_points(xy)` returns the normalized homogeneous points, whose last coordinate is exactly one, and the 3×3 similarity transform that moves the centroid to the origin and sets the mean Euclidean distance to $\sqrt{2}$. Coincident points must raise an error, not cause a division by zero.
- `estimate_homography_dlt(src_xy, dst_xy)` builds A with the column-major convention from A1, solves it with SVD, reshapes the result without transposing it, denormalizes, and returns a consistently scaled 3×3 homography. Reject a design matrix whose numerical rank is below eight.
- `symmetric_transfer_error(H, src_xy, dst_xy)` returns, for each correspondence i,

$$e_i = \tfrac{1}{2}\left(\|\pi(H\mathbf{x}_i) - \mathbf{x}'_i\|_2^2 + \|\pi(H^{-1}\mathbf{x}'_i) - \mathbf{x}_i\|_2^2\right),$$

  in pixels squared, where π converts homogeneous coordinates to Cartesian coordinates.

An invalid projection scale or a noninvertible map gives `inf` for the affected entries, never `nan` and never a clipped coordinate. Multiplying H by any nonzero scalar must leave every e_i unchanged. Scale the returned H so that `H[2,2]` is one when that entry is safely nonzero. Otherwise, return the version with unit Frobenius norm whose largest-magnitude entry is positive.

The tests check exact synthetic transforms, noisy overdetermined data, input order, scale invariance, unchanged inputs, accidental transposes, and degenerate configurations. You may use `numpy.linalg.svd`, but do not call an OpenCV homography estimator.

7.3 B3: RANSAC homography estimation (9 points)

Implement `ransac_homography(src_xy, dst_xy, threshold, confidence, rng)`, which returns `(H, inlier_mask, info)`. The threshold is in pixels and is squared before it is compared with the B2 error. Each trial must:

- sample four distinct correspondences without replacement
- reject repeated points and any collinear triple in either the source or destination sample
- score all correspondences with the symmetric transfer error
- update the exact finite-population budget with $\binom{k}{4}/\binom{N}{4}$ for the current consensus size k
- stop at that budget or at `RANSAC_MAX_TRIALS` trials, whichever comes first
- break ties by larger consensus, then lower median inlier error, then lower total inlier error, then the earlier trial
- re-estimate the final homography from every returned inlier

Use only the generator passed in `rng`, never the global random state. The same inputs and the same seed must give the same output.

`inlier_mask` is a boolean array of shape $(N,)$. `info` contains at least `trials` (an `int` that counts rejected degenerate samples), `adaptive_budget` (an `int`), `best_sample` (a tuple of four `int`, so two runs can be compared with ==), and `errors` (a `float64` array of shape $(N,)$ with each correspondence's error under the returned H). If the budget runs out before any four-point model reaches a consensus of four correspondences, raise `RuntimeError`. The supplied Part C cell catches this and marks the frame invalid.

Evaluate direct DLT and RANSAC on the supplied synthetic scenes: 100 points in a 640×480 image, noise $\sigma = 1$ px, outlier fractions 0, 0.2, 0.4, and 0.6, and seeds 16720 to 16724, with a 3-pixel threshold and confidence 0.99. Report the median corner-transfer RMSE, inlier precision and recall, and runtime. Most of the credit is for correct results and interpretation, not speed.

7.4 B4: Coarse-to-fine Lucas-Kanade refinement (11 points)

Implement `lucas_kanade_refine(template, image, p0, levels)`, which returns `(p, history)`.

Single-scale Lucas-Kanade linearizes image alignment around the current estimate, so it usually needs the remaining motion to be small. A coarse-to-fine version reduces a large full-resolution motion before applying the same local update. The notebook provides warping, interpolation, gradients, and Gaussian pyramids. Use the following procedure, where level 0 is full resolution and level $L-1$ is the coarsest level.

1. Build matching pyramids T^ℓ and I^ℓ with the supplied helper. Each level is half the size of the one before it in each dimension.
2. Write the translation at level ℓ as $p^\ell = [u^\ell, v^\ell]^T$. Convert the initial full-resolution translation $p_0 = [u_0, v_0]^T$ to the coarsest level: $p^{L-1} = p_0 / 2^{L-1}$.
3. At the current level ℓ, warp I^ℓ toward T^ℓ with the current translation and keep only pixels whose warped coordinates are valid. For each valid pixel x, form

$$r(x) = T^\ell(x) - I^\ell(x + p^\ell), \qquad A(x,:) = \begin{bmatrix} I_x & I_y \end{bmatrix}_{x+p^\ell}.$$

4. Solve the overdetermined system $A\,\Delta p \simeq r$ by least squares, update $p^\ell \leftarrow p^\ell + \Delta p$, and repeat until the update norm is below `LK_TOL` level pixels or `LK_MAX_ITERS` iterations have run. Do not form $(A^T A)^{-1}$ explicitly.
5. Stop a level early when fewer than `LK_MIN_VALID` pixels are valid in both images, when the least-squares rank is below two, or when the condition number of the design matrix exceeds `LK_MAX_CONDITION`. Record which of these happened.
6. If $\ell > 0$, start the next finer level with $p^{\ell-1} = 2p^\ell$ and refine again. The final answer is p^0, in full-resolution pixels.

`history` is a list of records that traces the search. It starts with the `level_start` record of the coarsest level and ends with the `level_end` record of level zero. Every record holds `phase` (`level_start`, `iteration`, or `level_end`), `level` (zero is full resolution), `p_level` (the estimate in that level's pixels), `p_full` (the same estimate times 2^ℓ), and `status` (`start`, `iterate`, `converged`, `max_iterations`, `singular`, `ill_conditioned`, or `insufficient_overlap`). Iteration records also hold `photometric_rmse`, the root-mean-square residual over that iteration's valid pixels. You may add more keys. The notebook cells read these by name, and they are what is graded.

As checks, confirm that a displacement of 8 pixels at full resolution becomes 2 pixels at pyramid level 2, verify the sign of the residual with a one-pixel synthetic translation, and report the estimate before and after refinement at every level. A single-scale run is the same algorithm with $L = 1$.

Test the method on synthetic translations and on the Oxford VGG `wall` sequence. The starter notebook downloads and checksum-verifies `wall` in its setup cell, or you can run `python data/setup_data.py --b4`. For the graded row, use `wall/img1.ppm` and `wall/img2.ppm` with `wall/H1to2p`, start from $p_0 = (0, 0)$, and crop the 64×64 region `x=68:132, y=268:332` from both images. The reference displacement is how the region's center moves under the given homography. Report the endpoint error, and the corner RMSE between the four translated region corners and their positions under the homography. Plot the fixed region and patch, the one-level and four-level photometric error by iteration, the reference and predicted quadrilaterals, and both geometric errors. If the download fails, the notebook runs a labeled synthetic check so the rest still works. That check earns no credit for this row.

Compare single-scale and coarse-to-fine refinement, and separate failures caused by a poor starting point from failures of brightness constancy. B4 studies translation only and assumes $\beta = 0$, so you do not need to estimate the A4 brightness term.

8. Part C (Create): Geometry-in-Motion Experience (20 points, 2.5 to 3 hours)

Generative AI is allowed for coding, debugging, visualization, interface work, and connecting approved pretrained matchers or flow models. You are responsible for understanding and testing everything you submit.

8.1 C1: Choose a planar target and application (4 points)

Using your own phone or camera, record a 5 to 15 second video of a clearly visible flat target, such as a poster, book cover, sign, painting, card on a table, or building facade. This recording is required. You may use downloaded or supplied videos, including the clips in `data/fallback/`, while you develop and debug, but they do not count for Parts C or D. The only exception is described in Section 10.

Choose one application:

- an augmented-reality overlay that stays attached to the plane
- stabilization that holds the plane still while the camera moves
- another application, approved by course staff, that visibly depends on planar motion estimation

Document the target, camera, capture location and setup, camera motion, lighting, frame count, and resolution, and explain why a homography is a reasonable model. Include one frame from the raw video and describe one way you expect the model to be violated.

8.2 C2: Build and improve the system (10 points)

Build the per-frame baseline from the supplied components: up to 2000 SIFT features, L2 two-nearest-neighbor matching with ratio 0.75, a mutual-consistency check, your normalized DLT RANSAC with a 3-pixel threshold and confidence 0.99, and frames resized so their longer side is at most 720 pixels. This processing size is separate from the 720p limit on your submitted video.

The baseline estimates a target-to-frame homography in every frame. The notebook provides the per-frame estimator, a default overlay renderer, and a short demonstration on a supplied clip. You write the logic that carries the result from one frame to the next.

A frame's estimate is valid only when all of the following hold:

- the homography is finite and invertible
- there are at least 10 inliers, and the inlier ratio is at least 0.25

- the convex hull of the inliers covers at least 15% of the target
- the square root of the median symmetric transfer error is at most 3 pixels, which is the same as a median B2 error of at most 9 pixels squared
- the projected target boundary is a finite convex quadrilateral

When a frame is invalid, keep showing the last valid result, labeled `STALE`, for at most three frames. After that, hide the overlay and search the whole frame again until a new estimate passes every check. Show one confidence score computed from the inlier ratio, the error, and the inlier coverage.

These validity checks, the three-frame `STALE` limit, and the whole-frame search are the required baseline, and they do not count as your improvement. For improvement credit, add one change that goes beyond the baseline and show its effect with a before-and-after comparison on your own video. For example:

- refine the feature-based homography with photometric alignment
- smooth the estimated motion without visible lag
- handle partial occlusion by requiring inliers spread across the target
- replace SIFT with an approved learned matcher
- compare a classical tracker with a modern correspondence or optical flow method

The final pipeline must produce a clean application view and a diagnostic view that shows correspondences and inliers, the tracked quadrilateral, confidence or error over time, and the current validity state.

Long videos run RANSAC once per frame, and a frame with no valid model uses the full trial budget. If processing is slow, keep every k-th frame with `read_video(..., frame_stride=k)`, or compute the target's features once with `extract_sift` and call `match_sift` for each frame. Neither changes the required geometry.

8.3 C3: AI-comprehension reflection (6 points)

In about 200 to 300 words, answer:

1. What did generative AI help you implement or explain?
2. How does the most important AI-assisted part work, including its inputs, outputs, and assumptions?
3. What did you change, debug, test, or redesign from the first generated version, and why?
4. What check would catch a result that looks right but is geometrically wrong?

If you did not use generative AI, write that. Then answer questions 2 and 3 about the most important part of your system that you wrote yourself or took from an allowed outside source, and how you changed it. You get the same credit either way.

9. Part D (Understand and Show): Evidence, Failure, and Communication (20 points, 1.5 to 2 hours)

Generative AI is allowed for visualization, with disclosure.

9.1 D1: Controlled robustness study (7 points)

Use your Part C video as the baseline. In the same session, record at least three short variants, changing one factor at a time: viewpoint, motion blur, lighting, partial occlusion, repeated texture, depth variation (parallax), less texture, or faster motion between frames. For each condition:

- describe what you kept fixed and what you changed, clearly enough for another student to repeat it
- write down your prediction before running the experiment
- mark the target's four corners by hand, in order, on 5 to 10 representative frames
- report the mean corner error or quadrilateral IoU against your marked corners, and a tracking measure such as the fraction of valid frames
- show one representative diagnostic image
- explain the result using sampling, conditioning, consensus, brightness constancy, the aperture problem, or model mismatch

Include a short table listing the target, camera motion, distance or viewpoint, lighting, changed factor, duration, frame count, and resolution for the baseline and each condition.

Two comparisons are required. At least one condition must compare direct DLT with RANSAC. At least one must compare single-scale with coarse-to-fine Lucas-Kanade, using your B4 function on a patch from your own video:

1. Pick two frames from one clip that are close enough in time that the patch moves no more than about 20 pixels in any direction. Mark the target's four corners in both frames. If you recorded the faster-motion condition, it is a good choice.

2. Estimate the homography from the first frame to the second from your marked corners, using your B2 function. This is the reference.
3. Choose a well-textured 64×64 pixel region inside the target in the first frame. Crop the same pixel window from both frames, convert both crops to grayscale in $[0, 1]$, and run `lucas_kanade_refine` with $p_0 = (0, 0)$, once with one level and once with four.
4. Report the endpoint error against the displacement of the region's center under the reference homography, and the corner RMSE against the region's corners mapped by that homography, as in B4.

A translation cannot follow a perspective change, so some corner error remains even when the center is found correctly. Say how much of your error comes from that. If a run stops with `insufficient_overlap`, the patch moved too far, so pick frames closer together.

Finally, revisit your prediction from A4. A low inlier error only shows that an estimate agrees with its own matches. It does not show that the estimate is correct, which is why the hand-marked corners are required.

9.2 D2: When a good-looking warp is wrong (5 points)

Choose one success and one failure from your video. For each, show the input frame, the inliers, the projected target boundary, the application output, and the synchronized error or confidence trace. Then explain:

1. whether the main cause is measurement noise, outliers, optimization, interpolation, or a scene that does not fit the model
2. why a result that looks right can still be geometrically wrong
3. what extra information, such as camera calibration, epipolar geometry, stereo, or more views, would be needed to support claims about the 3D scene

9.3 D3: Hero Result (8 points)

Make one visual result: a 10 to 30 second video, an animated comparison, or a small interactive web page. This is your Hero Result. Someone should be able to follow it without reading your code. It must include:

- the original video and your application result, shown together
- a synchronized diagnostic view of the estimated geometry
- one clear success and one failure or recovery
- a before-and-after comparison that shows your C2 improvement
- a short statement, visible in the result, of your assumptions and the method's limitations

Accepted formats are H.264 MP4 (yuv420p), GIF, or one self-contained HTML file. The limits are 30 seconds, 720p, 30 fps, and 75 MB. Also make one still image from the result, at least 1280 pixels on its longer side when your source allows. Section 12 says where to upload both and how to share the Hero Result on Slack.

10. Capture access, privacy, safety, and accessibility

Do not record people who have not agreed to it, private documents, personal information on screens, unsafe behavior, or places where recording is not allowed. Do not take risks to get a more dramatic shot. Keep your raw recordings on your own machine and submit only what is requested.

If an access, privacy, safety, or accessibility barrier stops you from recording, you may use the course-made clips in `data/fallback/` instead, but only with written approval from course staff given in advance. Ask before starting Part C, and no later than seven days before the deadline if you already know about the barrier. Follow `data/fallback/README.md`: contact the instructor and the PA3 lead privately, name only the category of barrier and the requirement it affects, and wait for an approval ID of the form `PA3-FB-YYYY-NNN`. You do not need to share a diagnosis or other personal details. Enter the ID in the notebook's approval cell, and list only the ID, the approval date, and the approving staff member's initials in your acknowledgments.

Approval only replaces your own recording. Everything else in Parts C and D is still required. Without a valid ID, the supplied clips are for debugging only.

11. Part E (Explore): Optional extension (up to +10 points, about 1 hour)

Generative AI is allowed, with disclosure. Try something beyond the required work. Some ideas:

- recover an essential or fundamental matrix and draw epipolar lines
- run calibrated stereo and show a disparity or depth map
- reconstruct and display a sparse 3D point cloud from two or more views

- compare Lucas-Kanade or Horn-Schunck with a modern learned flow model
- implement inverse-compositional alignment and measure how much faster it is
- add a parallax detector or a piecewise-planar model
- build an especially good AR, stabilization, or geometry-teaching interface

A meaningful extra exploration with a clear result and a short explanation earns up to $+5$. An especially thoughtful, technical, creative, or ambitious exploration earns up to $+10$ in total for Part E.

You can also earn $+1$ by sharing your Hero Result in the course showcase on Slack. Section 12 explains how. That point shares the 10-point cap with Part E.

12. Submission specification

Submit on Gradescope by **November 24, 2026, 11:59 pm ET**.

- Upload your compiled written PDF to the Gradescope assignment **PA3 Written**.
- Upload `pa3.py`, any other source files you wrote, the executed starter notebook with its outputs, your Hero Result, and its still image to **PA3 Code**.

Optional Slack showcase ($+1$). Sharing your Hero Result is optional. To take part:

1. Submit on Gradescope first. Posting on Slack does not replace the Gradescope upload.
2. Post your Hero Result file in the course Slack channel **`#showcase-hw3`**. If your Hero Result is an HTML page, you may post a link to it instead.
3. Post within 24 hours after the deadline.

Classmates vote with the `:movie_camera:` reaction for 72 hours. Posting earns $+1$ extra-credit point, and your combined extra credit is $\min(10,\ \text{Part E points} + 1)$. Votes never affect grades.

Use `answer_template.tex` without changing its labels, and start each labeled answer on the page break the template provides. In Gradescope, assign every page of each answer to its question and check the preview. Every required figure and table must appear both in your notebook and in the matching section of the PDF.

All required code must run in a fresh Google Colab runtime without paid compute, credentials, or private services. Do not submit API keys, access tokens, identifiable private images, or unnecessary raw video. Keep your raw recordings until grades are final.

The table below lists what each part needs.

Part	Written answers (PDF)	Figures and tables	Files (PA3 Code)
A1 to A4	Derivations and answers to every numbered question	None	None
B1	Answers B1.1 to B1.5	None	None
B2	Notes on your implementation and test results	None	`pa3.py`
B3	Interpretation of the results	Table of corner RMSE, precision, recall, and runtime for each outlier fraction	`pa3.py`, notebook
B4	One-level versus four-level comparison, and starting point versus brightness failures	Level checks, photometric error by iteration, error by level, and the `wall` figure	`pa3.py`, notebook
C1	Capture details, why a homography fits, and the expected violation	One raw frame	Notebook
C2	Baseline design and your improvement	Application view, diagnostic view, and before-and-after comparison	Notebook, other source files
C3	Reflection of 200 to 300 words	None	None
D1	Predictions, explanations, and A4 revisited	Capture table, corner-error table, DLT versus RANSAC, one-level versus four-level Lucas-Kanade, and one diagnostic per condition	Notebook
D2	Answers to the three questions	Success and failure panels with traces	Notebook

Part	Written answers (PDF)	Figures and tables	Files (PA3 Code)
D3	Where your assumptions and limitations appear	Still image	Hero Result, still image
Acknowledgments	Collaborators, sources, data, pretrained models, and AI use	None	None

13. Point summary

Part	Points
A1	9
A2	5
A3	5
A4	6
B1	5
B2	10
B3	9
B4	11
C1	4
C2	10
C3	6
D1	7
D2	5
D3	8
Base total	**100**
E and showcase combined	**0 to +10**